


# Project Report: Autonomous UAV Swarm Path Planning for Search and Rescue using Multi-Agent Reinforcement Learning

---

Thotakuri Guna Ranjan (22JE1019)
Potupureddi Dharmik (22JE0698)

**Guide - Prof. Chiranjeev Kumar**

---

# 1. Introduction

## 1.1. Motivation

In the immediate aftermath of a natural disaster, Search and Rescue (SAR) operations are fundamentally a race against time. The speed and efficiency of the response directly correlate to survivor outcomes. Traditional ground-based SAR teams are heroic but often constrained by hazardous, inaccessible terrain and limited situational awareness. Unmanned Aerial Vehicles (UAVs) have emerged as a critical technology to overcome these challenges. They offer rapid deployment, can scan vast areas quickly, and provide a constant flow of data from aerial sensors (e.g., thermal, optical), all while ensuring the safety of human responders.

However, the common operational model of one operator manually flying one drone is a significant cognitive bottleneck. This project is motivated by the need to move from manual operation to autonomous, *multi-UAV swarms*. The objective is to develop an autonomous *policy* that enables a team of UAVs to coordinate and explore an unknown area on their own, freeing human operators to focus on high-level strategy and victim identification.

## 1.2. Problem Statement

This project addresses the multi-agent path planning (MAPP) problem within an unknown and unstructured environment. The central challenge is:

*How can a swarm of multiple UAVs learn a decentralized, autonomous policy to efficiently and cooperatively achieve 100% coverage of an unknown, obstacle-filled disaster area, while simultaneously avoiding collisions with each other and the environment?*

This is not a simple navigation task from a known start to a known end. It is a complex exploration and information-gathering problem that requires dynamic, emergent coordination among all agents in real-time.

## 1.3. Objectives

The core objectives of this project are:

1. To design and implement a 2D grid-world simulation that models the key elements of an SAR scenario, including static obstacles and multiple UAV agents.

2. To formulate this cooperative exploration task as a Multi-Agent Reinforcement Learning (MARL) problem.
3. To implement and train a decentralized path-planning policy using the Proximal Policy Optimization (PPO) algorithm.
4. To design and iterate on coordination strategies, evolving from a static "home region" approach (main_2D.py) to a dynamic, shared "L fusion map" (MARL_PPO1.py).
5. To engineer a complex, multi-criteria reward function specifically designed to **minimize path redundancy and eliminate area overlapping** by penalizing revisited cells.
6. To **validate the proposed PPO-based approach by comparing its performance characteristics** (e.g., execution time, stability) against other existing algorithms discussed in the literature.

# 2. Literature Review / Background

## 2.1. Limitations of Classical Planners

Traditional path-planning algorithms are insufficient for our problem statement.

- **Classical Graph-Based Methods (A*, Dijkstra):** These algorithms are highly effective at finding the shortest, optimal path in a *known* environment. Studies show A* often outperforms other methods in computation efficiency and path quality. However, their fundamental limitation is the requirement of a complete, *a priori* map. In our SAR problem, the map is *unknown*; the mission *is* to create the map.
- **Heuristic & Metaheuristic Methods (PSO, GA):** Algorithms like Particle Swarm Optimization (PSO) and Genetic Algorithms (GA) are powerful optimizers. However, they are typically "offline" planners used to find an optimal path in a *static* environment. They are not designed for the rapid, reactive, real-time decision-making required in a dynamic multi-agent environment where the world state changes every second.

## 2.2. Justification for Reinforcement Learning

The limitations of classical planners led us to Reinforcement Learning (RL), which allows an agent to *learn* an optimal policy through trial and error.

- **Q-Learning vs. Deep Q-Networks (DQN):** Traditional Q-Learning uses a "Q-table" to store the value of every state-action pair. This is

computationally impossible for our 20x20 or 40x40 grids, as the number of possible states is astronomical. We therefore must use a **Deep Q-Network (DQN)**, which replaces the table with a neural network that *approximates* the Q-value, making the problem tractable.

- **Proximal Policy Optimization (PPO):** PPO is a state-of-the-art RL algorithm from the "Actor-Critic" family. It uses two neural networks: an **Actor** that decides which action to take, and a **Critic** that evaluates how good that state is. PPO is widely used because it is stable, simpler to implement than its predecessors (like TRPO), and provides robust performance.
- **Why Multi-Agent PPO (MAPPO)?** While PPO was designed for single agents, recent research has demonstrated its "surprising effectiveness" in cooperative multi-agent tasks. Benchmarks show MAPPO achieves performance *comparable or even superior* to more complex off-policy MARL algorithms like MADDPG and excels in *execution time*, which is critical for our rapid-response scenario.

# 3. System Design and Methodology

## 3.1. Algorithmic Approach: Multi-Agent PPO

Our system is built on a Multi-Agent Reinforcement Learning (MARL) framework, specifically using the **Proximal Policy Optimization (PPO)** algorithm. PPO is an on-policy, actor-critic algorithm that learns a policy for interacting with the environment.

- **Actor-Critic Model:** The system uses two distinct neural networks:
    1. **Actor (The Policy):** This network takes the current state (observation) as input and decides which action to take (e.g., move up, down, left, right).
    2. **Critic (The Value Function):** This network also takes the state as input but outputs a single scalar value, which is an *estimate* of the total future reward the agent can expect to get from that state.
- **Why PPO?** We chose PPO for its balance of simplicity, stability, and performance. Its key innovation is a **clipped surrogate objective**. This mechanism prevents the policy from changing too drastically in one update, which avoids the catastrophic performance collapse common in other algorithms and leads to more stable and reliable training. In multi-agent settings (MAPPO), this stability is crucial, and PPO has been shown to

achieve results competitive with, or even superior to, more complex off-policy methods.

## 3.2. Evolution of Coordination Strategy

Our methodology evolved through two distinct phases as we searched for a more robust and effective coordination strategy.

#### 3.2.1. Initial Approach: Static "Home Regions" (main_2D.py)

Our first implementation, found in main_2D.py, was a system with **3 UAVs** operating in a **20x20 grid**.

- **Coordination Strategy:** To enforce cooperation and guarantee full coverage, we employed a simple, static division of labor. The 20x20 map was pre-divided into 3 distinct "home regions."
- **Methodology:** Each of the 3 UAV agents was assigned one home region at the beginning of the episode. The reward function was engineered to strongly incentivize agents to explore *within their assigned region* and penalize them for leaving it.
- **Limitations & Rationale for Change:** This approach, while simple to implement, proved to be highly inflexible and suboptimal for a dynamic SAR scenario.
    1. **Inefficient:** If one UAV's region was small or had few obstacles, it would finish quickly and become idle, while another UAV might be stuck with a large, complex area. This led to poor load balancing.
    2. **Not Robust:** If one UAV failed (e.g., was "destroyed" in the simulation), its entire region would remain unexplored.
    3. **Not Truly Cooperative:** The agents were not *cooperating*; they were executing independent, parallel tasks. This rigid, non-adaptive strategy led us to discard the "home region" concept.

#### 3.2.2. Final Approach: Dynamic "L Fusion Map" (MARL_PPO1.py)

We re-architected the entire system to foster true, *emergent* cooperation. This final model, defined in MARL_PPO1.py, uses **6 UAVs** in a **40x40 grid**.

- **Coordination Strategy:** We completely removed the "home regions." Instead, all 6 UAVs operate in the same shared 40x40 space and contribute to a single, collective set of shared-knowledge variables, which we term the "L Fusion Map."

- **Methodology:** Cooperation is no longer hard-coded. It is an *emergent behavior* driven entirely by the reward function. This is enabled by key global variables that all agents contribute to and are judged against:
    1. self.global_explored_cells: A set containing every (x, y) coordinate that *any* UAV has visited.
    2. self.visitation_counts: A grid that tracks how many times *any* agent has visited *each* cell.
- **Why This is Superior:** This design is dynamic, robust, and scalable.
    1. **Dynamic Task Allocation:** If one part of the map is fully explored, the agents who were there are now free to help explore other, more complex areas, ensuring all 6 agents remain productive.
    2. **Emergent Cooperation:** The reward function creates a powerful economic incentive. An agent gets a large reward for visiting a new cell (+15.0) but is *penalized* for moving to *any* cell that has already been visited (-0.05 * self.visitation_counts). To maximize its *own* score, each agent *must* seek out unexplored territory and avoid its teammates. This naturally forces the swarm to spread out and cover the map with maximum efficiency, **effectively removing path redundancy and minimizing area overlapping**.

### 3.3. Communication Model

The coordination in our final approach (MARL_PPO1.py) is predicated on a specific communication model.

- **Model:** The system does *not* simulate a limited or unreliable communication range. Instead, it assumes a **perfect, global, and instantaneous communication range** where all agents have real-time access to all collective information.
- **Implementation:** This is implemented via the central simulation environment acting as an information hub.
    1. At each step, the environment gathers the actions from all 6 agents.
    2. It updates the global state variables (like self.global_explored_cells and self.visitation_counts) based on the joint actions of all agents.
    3. The observation-building function then uses this *perfect, global* data to construct the 9-channel "L Fusion Map" (observation tensor) for each individual agent.

4. Therefore, every agent receives a perfect, real-time map that includes the position of all teammates (teammate_pos_view) and the entire swarm's collective knowledge (global_explored_view, global_frontier_view).

### 3.4. Tools and Technologies

The project was implemented in Python using a standard set of data science and deep learning libraries:

- **torch (PyTorch):** The primary deep learning library used to build the Actor and Critic neural networks and run the PPO optimization.
- **numpy:** The fundamental library for scientific computing, used for all grid-world representations, position calculations, and observation data.
- **matplotlib.pyplot:** Used to render and visualize the environment, agent paths, and coverage.
- **heapq:** A standard Python library used to implement an efficient A* search algorithm. This is used internally by the environment to calculate the global_frontier.

## 4. Implementation Details

### 4.1. Simulation Environment Implementation (MARL_PPO1.py)

The core of the project is the custom simulation environment defined in MARL_PPO1.py.

- **Parameters:** The environment is initialized with grid_size = 40 and num_uavs = 6.
- **Key State Variables:** The environment maintains a comprehensive global state:
    - self.grid: The 40x40 numpy array representing the static world (0 for empty, 1 for obstacle).
    - self.global_explored_cells: A set tracking all visited cells for coverage calculation and new cell rewards.
    - self.global_frontier: A set of all explorable "edge" cells, computed via heapq (A*), used for the frontier navigation reward.
    - self.visitation_counts: A 40x40 numpy array to track visit frequency for the revisit penalty.

- self.stuck_counters: A dict tracking each agent's idleness to apply the stuck penalty and efficiency bonus.

### 4.2. Observation Space: The "L Fusion Map"

The observation function constructs the "L Fusion Map" for each agent. This is a comprehensive, image-like torch tensor with dimensions (9, 40, 40). This "global view" is given to each agent and contains the following 9 channels:

1. **global_obstacle_view**: A 40x40 binary map showing all obstacles discovered by *any* agent.
2. **target_view**: A 40x40 binary map (legacy from main_2D.py) showing undiscovered targets.
3. **teammate_pos_view**: A 40x40 binary map marking the current positions of all *other* 5 UAVs.
4. **my_pos_view**: A 40x40 binary map marking the agent's *own* current position.
5. **global_explored_view**: A 40x40 binary map showing all cells in self.global_explored_cells (the collective memory of the swarm).
6. **global_frontier_view**: A 40x40 binary map marking the cells in self.global_frontier.
7. **visitation_view**: A 40x40 map of the visitation_counts, normalized to provide a "heatmap" of swarm activity.
8. **state_view**: A 40x40 map filled with a constant value (0.5), representing the "exploration" mode.
9. **time_view**: A 40x40 map filled with the normalized global step counter (global_step_counter / 2200.0), giving the agent a sense of time.

### 4.3. Multi-Criteria Reward Function (MARL_PPO1.py)

The emergent behavior is shaped by the complex, multi-criteria reward function defined in the simulation's step method. This function is calculated for *each agent* at *each time step*:

| Reward / Penalty | Value | Purpose |
|---|---|---|

| | | |
|---|---|---|
| **New Cell Discovery** | +15.0 | The primary objective. Enormous reward for visiting a cell not in global_explored_cells. |
| **Frontier Navigation** | +5.0 | A "shaping reward" for moving closer to the global_frontier. Guides agents toward the unknown. |
| **Efficiency Bonus** | +2.0 | An extra reward if a new cell is found within 10 steps of the last one, encouraging momentum. |
| **Obstacle Collision** | -3.0 | Large penalty for hitting a wall. Teaches environmental safety. |
| **Agent Collision** | -2.0 | Large penalty for moving to the same cell as a teammate in the same step. Teaches social spacing. |
| **Revisit Penalty** | -0.05 * self.visitation_counts | A *dynamic* penalty that scales with visits. This is the key for **removing path redundancy**. |
| **Stuck Penalty** | -0.1 | A small penalty for any step that does *not* result in a new cell. Punishes inefficient wandering. |

| | | |
|---|---|---|
| **Step Penalty** | -0.005 | A tiny "cost of living" penalty. Encourages the swarm to finish the mission as *fast* as possible. |
| **Target Finding** | +5.0 | A de-prioritized reward (legacy from main_2D.py) for finding specific targets. |

### 4.4. Agent and Network Architecture (MARL_PPO.ipynb)

The agent's "brain" is defined in MARL_PPO.ipynb as an ActorCritic neural network class using PyTorch.

- Network Architecture: CNN
  Because the observation space from MARL_PPO1.py is an image-like (9, 40, 40) tensor, a simple Multi-Layer Perceptron (MLP) is insufficient. The network architecture must be a Convolutional Neural Network (CNN), which is designed to process spatial data.
  1. **Actor (Policy Network):** The Actor is a CNN that takes the (9, 40, 40) "L Fusion Map" tensor as input. It passes this through several convolutional layers to extract spatial features, flattens the result, and finally passes it through fully-connected layers to output *logits* for the 5 discrete actions (Up, Down, Left, Right, Stay).
  2. **Critic (Value Network):** The Critic is a separate CNN (or shares the convolutional base with the Actor) that also takes the (9, 40, 40) tensor as input. It processes the map and outputs a *single scalar value* representing the estimated future reward ($V(s)$) from that state.
- **PPO Training Loop:**
  1. **Data Collection (Rollout):** The 6 agents (using their current Actor policy) are run in the simulation environment for a set number of steps (e.g., 2048 steps). All collected (state, action, reward, next_state, done) transitions are stored.
  2. **Advantage Calculation (GAE):** At the end of the rollout, the Critic network is used to calculate the "advantage" for each step taken (i.e., "was this action better or worse than expected?").
  3. **Update Epochs:** The algorithm then iterates over the entire "rollout" dataset multiple times (epochs). In each epoch, it updates the Actor

and Critic networks using the PPO **clipped surrogate objective**. This update rule ensures the policy improves without changing too much, guaranteeing stable learning.

# 5. Results and Analysis

## 5.1. Performance Metrics (KPIs)

A quantitative analysis of the trained policy is based on key performance metrics (KPIs) derived directly from the project's objectives and reward function.

1. **Area Coverage Rate (%):** The primary mission metric. This is the final size of self.global_explored_cells as a percentage of all non-obstacle cells. A high value (near 100%) indicates a successful exploration, driven by the +15.0 reward.
2. **Time to Completion (Steps):** Measures the operational *speed* of the swarm. This is the total number of environment steps required to achieve 100% coverage. A lower step count is better, driven by the -0.005 step penalty.
3. **Collision Rate (per Episode):** The total number of agent (-2.0) and obstacle (-3.0) penalties received per episode. A successful policy will learn to minimize this count to near-zero.
4. **Exploration Redundancy Rate:** This metric, defined as (Total Steps Taken by all Agents) / (Total Unique Cells Explored), measures the *efficiency* of the swarm. A rate of 1.0 is a perfect (but impossible) score. A low rate proves that the -0.05 * self.visitation_counts penalty was successful in **preventing area overlapping and removing path redundancy**.
5. **Average Reward:** The core RL metric, showing the agent's ability to maximize its objective.

## 5.2. Validation and Comparison with Existing Algorithms

Validating the choice of PPO is a key part of the project. The proposed approach is validated by comparing its known performance characteristics against the other primary algorithm classes.

- **vs. Classical Planners (A*, RRT*):** As established in the literature review, classical planners are not suitable for this *problem definition*. A* is optimal for finding a path on a *known* map, but it cannot perform *exploration* of an *unknown* map. Therefore, a learning-based approach like PPO is required.

- **vs. Other MARL Algorithms (e.g., MADDPG):** This is the most critical comparison for validation. Research benchmarking MARL algorithms for UAV swarms provides clear insights.
    - **Performance:** Studies show that MAPPO (our approach) achieves "surprisingly strong performance" and delivers results that are *comparable or even superior* to complex off-policy methods like MADDPG.
    - **Execution Time:** In comparative evaluations of algorithms for multi-UAV navigation, PPO and MADDPG were both shown to **excel in execution time** (mission completion speed), a critical KPI for our SAR scenario.
    - **Stability & Accuracy:** PPO is often cited for its **stable and accurate convergence** and is simpler to implement and tune than its counterparts.

This comparison validates that PPO is not a lesser choice, but a robust, high-performance, and stable baseline for this type of cooperative MARL problem, justifiying its selection for the project.

# 6. Limitations and Future Enhancement

## 6.1. Limitations of the Proposed Solution

1. **Sim-to-Real Gap:** This is the most significant limitation. The project is a 2D numpy grid simulation (main_2D.py, MARL_PPO1.py). A policy trained in this perfect, discrete world will not transfer directly to a physical drone. It does not account for real-world physics, momentum, battery life, or sensor noise.
2. **Communication Model:** The "L Fusion Map" (MARL_PPO1.py) relies on a perfect, global, real-time communication link. In a real disaster, communication is unreliable or non-existent, making this centralized knowledge source a critical failure point.
3. **Scalability:** The centralized global state (e.g., self.global_explored_cells) creates a bottleneck. As the number of agents and map size increases, managing this global state becomes computationally prohibitive.

## 6.2. Future Enhancement

1. **High-Fidelity 3D Simulation:** The first step is to migrate the project from a 2D numpy grid to a 3D physics-based simulator like **AirSim** or **Gazebo**. This would allow for training on continuous control and realistic flight dynamics.

2. **Bridge the Sim-to-Real Gap:** Implement **Domain Randomization**—adding noise to actions, sensor readings, and physics in the simulation. This forces the PPO policy to become more *robust* and less reliant on a perfect environment, making it more likely to work on real hardware.
3. **Decentralized Communication:** Replace the global "L Fusion Map" with a decentralized mapping protocol. Agents would have a *limited communication range* and would need to learn to *explicitly* share and merge their *local* maps with teammates when they are in range.

## 7. Conclusion

This project successfully designed and implemented a Multi-Agent Reinforcement Learning system for autonomous UAV swarm exploration. We demonstrated a clear methodological evolution, advancing from a simple, static "home region" coordination strategy (main_2D.py) to a far more robust and dynamic *emergent-cooperation* model (MARL_PPO1.py).

By engineering a sophisticated multi-criteria reward function and leveraging the stability and performance of the MAPPO algorithm, we developed a policy that learns to balance exploration speed, agent safety, and swarm efficiency. The "L Fusion Map" concept, implemented as a 9-channel global observation tensor and driven by shared variables like self.global_explored_cells and self.visitation_counts, proved highly effective at forcing the swarm to spread out and cover the environment with minimal **path redundancy and area overlapping**. While limited by its 2D, simulation-based nature, this project provides a strong and validated foundation for future research into autonomous systems for real-world search and rescue.